

Primitive data types

Primitive data types are the basic data types in C used to store different kinds of values, such as integers, decimal numbers, and characters.

Types of Primitive Data Types

Data Type	Description	Example
int	Stores whole numbers	int age = 20;
char	Stores a single character	char grade = 'A';
float	Stores decimal numbers (single precision)	float price = 25.5f;
double	Stores decimal numbers (double precision)	double pi = 3.141592;
void	Represents no value	void display();

Typical Size and Range

The sizes below are common on modern systems, but the C standard allows some variation depending on the compiler and architecture.

Data Type	Typical Size	Example Range
char	1 byte	−128 to 127 (signed)
int	4 bytes	−2,147,483,648 to 2,147,483,647
float	4 bytes	Approximately 6–7 decimal digits of precision
double	8 bytes	Approximately 15–16 decimal digits of precision
void	No storage for an object	Not applicable

C Program to Demonstrate Primitive Data Types

```
#include <stdio.h>
int main()
{
    int age = 20;
    char grade = 'A';
    float mark = 89.5f;
    double salary = 45678.9876;
    printf("Age = %d\n", age);
    printf("Grade = %c\n", grade);
    printf("Mark = %f\n", mark);
    printf("Salary = %lf\n", salary);
    return 0;
}
```

```
Age = 20
Grade = A
Mark = 89.500000
Salary = 45678.987600
```

Program to Find the Size of Data Types

The sizeof operator is used to determine the memory occupied by a data type or variable.

```
#include <stdio.h>
int main()
{
    printf("Size of int = %zu bytes\n", sizeof(int));
    printf("Size of char = %zu byte\n", sizeof(char));
    printf("Size of float = %zu bytes\n", sizeof(float));
    printf("Size of double = %zu bytes\n", sizeof(double));
    return 0;
}
```

Size of int = 4 bytes
Size of char = 1 byte
Size of float = 4 bytes
Size of double = 8 bytes

Types of **int** data type

Type	Meaning	Typical Size	Typical Range
short int	Small integer	2 bytes	−32,768 to 32,767
unsigned short int	Small positive integer	2 bytes	0 to 65,535
int	Normal integer	4 bytes	−2,147,483,648 to 2,147,483,647
unsigned int	Positive integer only	4 bytes	0 to 4,294,967,295
long int	Large integer	4 or 8 bytes	Depends on system
unsigned long int	Large positive integer	4 or 8 bytes	Depends on system
long long int	Very large integer	8 bytes	−9,223,372,036,854,775,808 to 9,223,372,036,854,775,807
unsigned long long int	Very large positive integer	8 bytes	0 to 18,446,744,073,709,551,615

```
#include <stdio.h>
int main()
{
    short int a = 100;
    int b = 1000;
    long int c = 100000;
    long long int d = 10000000000LL;
    printf("short int    = %hd\n", a);
    printf("int          = %d\n", b);
    printf("long int       = %ld\n", c);
    printf("long long int = %lld\n", d);
}
```

```
    return 0;
}
short int    = 100
int          = 1000
long int     = 100000
long long int = 10000000000
```

Key Points

- ✓ Primitive data types are the building blocks of C programming.
- ✓ Each data type has a specific purpose and memory requirement.
- ✓ The sizeof operator helps determine the memory occupied by a data type.
- ✓ The format specifier must match the type of value being printed.